https://www.perplexity.ai/search/okay-above-is-an-appengineerin-2KzF0fR5SI6L3IH4nUCENw

BLUF: Single Next.js 16 monorepo with Prisma/PostgreSQL and server-side rendering. The build has a strict sequential bottleneck - auth, DB schema, and RBAC gates must be locked before any feature work starts, since parallel builders will break against schema drift without CI/CD. Three streams can then run in parallel (CV/profile, job board, admin panel shells), but business logic - particularly recruiter search and the admin recommendation gateway - can only be wired after consent placeholder and tier gates are in place. The recommendation gateway is the highest-complexity V1 feature: it's a 3-layer dependency (consent check → Gold/Platinum tier gate → per-recruiter data isolation) and introduces a new DB access pattern not yet explicitly in the schema from Report 3.

Sequential Bottleneck - Plain Terms

Without CI/CD, there's no automated check that everyone's code is pointing at the same database shape. If one developer adds a consent_status field to the Prisma schema but hasn't pushed it, and another developer builds a page that queries that field from their local DB where the column doesn't exist yet - it silently breaks locally and nobody catches it until a git collision. Auth, DB schema, and tier gates have to be finalised first so every parallel builder is coding against an identical, stable data contract. Changes after that point require a coordinated schema migration push to all local envs before anyone continues - which is the "migration runbook per release" risk flagged in Report 1.

Option A - Admin Recommends Students to Recruiter: 3 Approaches

A1 - In-app flag + recruiter tab *(recommended)*
Admin filters consented students by enum fields (location, degree, yrs exp), selects n students, tags them as recommended to firm X (or multiple firms independently). Each firm sees a "Recommended Students" tab in their dashboard showing only their own list - no cross-firm visibility. Admin can remove the tag to revoke. DB cost: one admin_recommendations table (student_id, firm_id, admin_id, created_at, revoked_at). Fits existing schema pattern from Report 3, fully in-app, revocable, privacy-isolated by firm_id join.

A2 - CSV export only *(simplest, lowest dev cost)*
Admin filters same enums, hits "Export CSV" for firm X. No in-app recruiter view added. Revoke is manual (email). Zero DB changes needed beyond consent check. Downside: no audit trail, not revocable in-platform, client likely won't accept as final but could ship as interim.

A3 - Student-initiated referral request
Student requests to be surfaced to firm X → triggers admin review task → admin approves/denies → firm can see that student's profile. Consent already implicitly granted via the request. Inverts control to students which is philosophically aligned with the platform's consent-first model, but adds a 3-step async flow and a notification/queue system not currently in scope - highest complexity of the three.

Recommendation: go with A1. One new table, minimal schema delta, fully in-app, revocable, clean privacy isolation per firm. A3 is philosophically better but over-engineered for V1.

Option B - Admin Surfaces Firms to Students

Admin marks a Gold/Platinum firm as "recommended for student cohort X" (e.g. CS students, London-based). Students meeting the criteria see that firm ranked first / in a "Recommended" section in their opportunities/job board view - implemented as a

firm_priority flag or ordering weight in the existing job listings query join. No new tab, no new surface, just amended ORDER BY or a join weight. One DB column addition on companies table (admin_recommended: boolean), or a lightweight junction table if cohort targeting is needed. Privacy-safe: students don't see each other, firms don't see targeting criteria.

B is significantly simpler than A. If you need to push back on one, push back on B or ship it as a trivially small bolt-on after A1 is done.

Next steps:

(Check scaffold readmes + review code sections to have better understanding of existing build)

Work out how to set up local

Fork existing scaffold and run are own tests on it the point is to understand local instance
first to own repo - **don't merge, dont raise any PRs to existing scaffold**

Postgresql

Nextjs setup

Create dummy button

On click of button insert new user in database (check if prisma is updated)

Frontend -> nodejs layer -> actually populates db correctly

Updated End-to-End Requirement Groupings

Key: F=Foundation, CON=Consent, CVP=CV & Profile, JOB=Job Board, SRCH=Recruiter Search, ADM=Admin Panel, REC-A=Recommendation Gateway A (admin→recruiter), REC-B=Firm Surfacing B (admin→student), NOTIF=Contact & Notification, DASH=Admin Dashboard. G/P=Gold/Platinum tier only. L1-L5=sequential build layer. "Consent-gated"=only consented students visible to recruiter.

Takeaway from Layer 1: login page most important, Sadhana@ to own full stack of login frontend creating all components + inserting & fetching from DB.

ID	Short Description	Group	Layer	Key Dependency	Notes
SETUP-1	Login page UI + Next.js routing shell	F	L1	None	We need to check, login page UI/logic is already there in existing codebase quality needs to be verified, potential refactoring needed.
SETUP-2	PostgreSQL schema + Prisma migrations + local dev runbook Daniel/Client previously recommended working directly in the existing scaffold github project. However, he has existing main protection permissions enabled. This means there would be a	F	L1	None	Prisma file schema database in existing codebase is quite limited components we work on may need updates/alignment with wider dev efforts of the group over time. Requires close review during development, individual devs to keep in mind, refactor if needed and share learnings with the wider team. user/member roles (tiered across platinum/gold/silver/bronze)

	dependency on him reviewing our pull requests - inducing the risk of stale DB schema per user (due to parallelised work, e.g user x write change <> Daniel approval delay <> user y receives/accesses user x's write).**Solution:** clone existing repo, removing dependency. This could result in conflicts if merging back to original scaffold repo, increasing work on Daniel's end. Solution is to use our cloned repo instead for UCL virtual machine (VM) deployment. Validating with Daniel 4-March.				already exist in codebase. This means we can use those endpoints for development, but need to ensure access persists across new features we build (e.g. keeping in mind not all users will access CV library). Therefore some logic will need to be built on top of this (bottom-up), and overarching orchestration needs to enforce across project (top-down). Potentially will need to create new DBs to handle this (to be reviewed/implemented during development work. Summary: ie 1. Use existing DB schema where relevant 2. add new tables where needed.
FR-S-1 / FR-R-1	Student & recruiter registration (UCL email / verified company domain)	F	L1	SETUP-2	We need to define how to verify company domain integration with UCL UCL likely already has verified domain list, but we'd need to coordinate with UCL to get pre-existing data, and introduces **risk** of only being able to test with verified ucl domains, reducing speed of development. Solution for starting development is to handle via dummy list of domains, and introduce real-world ucl verification at later stage in project **or** leave open-ended for handover to Daniel/Client/UCL IT. **Discuss with Daniel 5-March.**
NFR-13	UCL SSO integration (Shibboleth/SAML reuse from scaffold)	F	L1	SETUP-2	SSO is already in scaffold, we can inherit this with minimal changes **We'll pick up a separate, but related discussion on extending admin scope to approve/verify new recruiter domains not already in ucl verified domain list mid-project as a group.** **Solution for starting development: develop ui, leave sso out of it /keep placeholder** On successful login during development assign student/recruiter/admin roles to test/verify page/feature access logic. This results in easier testing, quicker development **Existing SSO code quality needs review,** at beginning of the project.
FR-S-2	2FA for all accounts	F	L1	FR-S-1 <-	2FA is already built-in via SSO integration BUT we don't know if Daniel has implemented it for this specific project. Needs review, we should be able to handle it quick
RBAC-1	Tiered permission gates at component level (bronze/silver/G/P/student/admin)	F	L1	FR-S-1	1. pull individual recruiter type RBAC in scaffold (existing repo) other components are not. There is a different project repo within scaffold called talent discover. Here other tier role-based logic is already defined - but we need to find a way to implement it elegantly.

					Overarching Platinum/gold/silver/bronze access logic **should** already be there, but we need a dev to review in-depth and validate. 2. implement sub-page level logic in between sub user pages <- we know it works if local DBs during operations work.
FR-S-13	Student consent toggle skeleton (binary provide/withdraw, placeholder logic)	CON	L2	FR-S-1, SETUP-2	We'll add a placeholder for this during development. We don't need to worry about gdpr/consent management, this will be handled via ucl IT. Solution: in Student DB add flag which companies given consent to view query fetch only that particular company flag enabled/disabled <- if they add on top of that they'll do it later on
FR-S-12	Student whitelist/blacklist per-company visibility controls	CON	L2	FR-S-13	write boolean logic/flags. Direction (requires further review) -> need to think from DB perspective access student id join company ID get appropriate state to serve blacklist same join operation fetch join + mark table column with new write value
FR-S-6	Student profile fields (skills, experience, certifications, projects)	CVP	L2	SETUP-2, RBAC-1	
FR-S-11	Link external profiles (LinkedIn, GitHub)	CVP	L2	FR-S-6	
FR-A-1	Admin: approve/reject company registration	ADM	L2	RBAC-1, FR-R-1	Behaviour: company registers, request goes to admin for approval UI A: request status B:once approved by admin create new entry for company and give appropriate role (tiered) (update role of user x in backend) initially accessible to every student <> blacklist specific companies therefore default value to True
FR-R-14	Recruiter: post job (title, location, salary band, role type, description)	JOB	L2	RBAC-1, FR-R-1	block storage needs to be constructed back in production in postgresql usually handled externally blob (binary large objects) -> check with Daniel aws storage later on as plan b even in case of nosql still converted + pieced back together has less efficiency Storing images + doc behaviour needs to be defined + implemented elegantly (might not be straightforward in case of cloud set-up, postgresql storage not ideal/viable)

FR-S-4	Student: upload CV in PDF/DOCX with file storage	CVP	L3	SETUP-2, FR-S-6	
FR-S-5	Student: update/delete/replace CV	CVP	L3	FR-S-4	storage part of it needs research To discuss 2-3 approaches store in postgresql or external with Daniel Potential simplified solution is create public gdrive links for testing etc + can be valid workaround for our v1 solution update delete replace is same <- needs research on how we handle actual underlying data storage same as before
FR-S-7	CV keyword tagging	CVP	L3	FR-S-4	normalise tags single big string ties to actual feature we want to build this out for <- not defined BUT not ML related for target CV tagging we can potentially push to stretch feature considering it's not directly NEEDed for other components, but beneficial for admin 'recommendation' filter mechanism
FR-S-8	Multiple CV uploads (e.g. ML-focused vs systems-focused)	CVP	L3	FR-S-4	
FR-S-3	Deleting account	CON	L3		direct CRUD op to should be straightforward
FR-A-2	Admin: suspend/ban students or companies	ADM	L3	FR-A-1	separate access table should be path forward. <- vs normalising only admin has access to - creates redundant table when banning/suspending incorporating in the current user table isa risk because if a student is marked as suspended, it will suspend students from other related applications = if we expand and build multiple sub applications over time (post handover), suspending in 1 core student table would suspend the same student across the ecosystem. therefore we add a new <<separate>> table, instead writing to parent student table.
FR-R-3	Recruiter search students by skills, exp, location (consent-gated, Gold/Premium only)	SRCH	L4	FR-S-13, FR-S-6, RBAC-1	location is straightforward + type of degree skills could tie to cv tagging but these extra enums/components adds complexity and might not be needed
FR-A-7a	Admin filters consented students by enumeration (location, degree studied → CV tags (granular) to firm(s) **(this intermediate step is skippable, we will/can work to simplify)** → "Recommended Students" tab in recruiter view; per-firm isolation; revocable (A1 approach)	REC-A	L4	FR-S-13, RBAC-1, FR-A-1, new admin_recommendationstable	

FR-A-7b	Admin revoke/amend recommendation access (remove tag, access disappears from recruiter tab)	REC -A	L4	FR-A-7a	
FR-A-8	Admin recommends Recruiter to student -> Admin flags G/P firm as relevant to student cohort; surfaces firm higher in student job/opportunity view via join weight or ordering (B approach)	REC -B	L4	FR-R-14, FR-S-13, FR-A-1	We're looking to cut admin recommends recruiter to student, adds unneccessary complexity with not much value considering 1-to-**many** recruiter:student ratio.
DASH-1	Admin dashboard: consented vs total student count, approved recruiter count, matchable pair count - pie/donut visualisation only, no usage telemetry	DA SH	L4	FR-S-13, FR-A-1	North star metric-> What users passed initial verification, and can use platform without friction? Eg. student passed consent verification + recruiter approved
DASH-2	Admin dashboard (stretch): add filters by degree	DA SH	L5	DASH-1	To be defined in the future if needed
FR-R-8	Recruiter contacts student via email (external, link/mailto only, not in-platform)	NO TIF	L5	FR-R-3, RBAC-1	We will handle this simply, potentially with status message confirming comms is handled outside of platform / tooltips in UI if beneficial.
FR-S-17	Student notified when company initiates contact request	NO TIF	L5	FR-R-8	Default to email trigger. Potentially in app, e.g. build notification drop-down tab (or company x tried to context you bell icon exclusively for this type of notification for student)

—----------

The core challenge is that business logic spans multiple pages owned by different people - so the pattern to adopt is **contracts first, implementation second**. Here's how to think about it at a high level:

The Core Mental Model

Treat the orchestration layer as a shared "engine" that sits below all the pages. Individual subpage builders never write business logic themselves - they call functions from the shared engine and get back typed results. The engine is owned by one person (you or tech lead), subpages just consume it.

text
```
/src
├── app/                  ← individual owners build here (pages, layouts)
│   ├── student/
│   ├── admin/
│   └── recruiter/
├── lib/                  ← shared engine, one owner, no parallel edits
│   ├── services/         ← business logic lives here
│   ├── auth/             ← RBAC + tier gates
│   └── db/               ← Prisma client wrapper
└── types/                ← shared TypeScript contracts
```

The `/lib` boundary is the critical discipline - parallel builders import from it, never bypass it with direct Prisma calls or inline logic.[osohq]

Step 1 - Define Type Contracts Before Anything Else

Before anyone writes a page, the types file defines the shape of every entity shared across components. This is the "stable data contract" that prevents parallel builders from inventing their own data shapes:[prisma]

typescript
```typescript
// types/index.ts
export type UserTier = 'bronze' | 'silver' | 'gold' | 'platinum' | 'student' | 'admin'

export type ConsentStatus = 'pending' | 'consented' | 'withdrawn' // stub for now

export type StudentProfile = {
  id: string
  tier: UserTier
  consentStatus: ConsentStatus
  skills: string[]
  // etc.
}
```

Once these are locked, parallel builders program against the types even if the real data isn't there yet. Changing a type becomes a deliberate coordinated action, not an accident.

Step 2 - Stub the Service Layer First

Each business logic function is created as a stub returning dummy data. Parallel builders call the real function signature - when the tech lead wires the real logic later, nothing in the pages needs to change:[osohq]

typescript
```typescript
// lib/services/consent.ts
export async function getConsentStatus(studentId: string): Promise<ConsentStatus> {
  // STUB - replace with real Prisma query when consent schema locked
  return 'consented'
}

// lib/services/recommendations.ts
export async function getAdminRecommendations(firmId: string): Promise<StudentProfile[]> {
  // STUB - returns empty array until admin recommendation gateway built
  return []
}
```

Pages call `getConsentStatus()` from day one. When real logic lands, pages get real data with zero refactoring.

Step 3 - RBAC as Middleware + Component Gate

Two-layer approach for tier gates so parallel builders can't accidentally expose restricted content:[osohq]

1. **Route level** - `middleware.ts` at project root blocks unauthenticated/wrong-tier users before the page renders. One person writes this once.
2. **Component level** - a `<TierGate tier="gold">` wrapper component each builder drops around restricted UI sections. Stub now, real logic wired later.

typescript
```typescript
// components/TierGate.tsx
export function TierGate({ tier, children }: { tier: UserTier, children: React.ReactNode }) {
  const { userTier } = useAuth() // stub returns 'gold' during dev
  if (!hasAccess(userTier, tier)) return null
  return <>{children}</>
}
```

Every parallel builder wraps restricted sections in `<TierGate>` from day one. The `hasAccess` logic gets filled in when RBAC is fully wired - component usage never changes.

Step 4 - Orchestration Sequence for Parallel Work

The dependency order maps directly to your build layers:

1. **Tech lead / you lock first** - `types/index.ts`, `lib/db/`, `lib/auth/`, `TierGate` stub, all service stubs in `lib/services/`. This is a 1-2 day upfront investment before anyone else starts.
2. **Parallel builders start** - they import services and types, build page shells that call stub functions. Their work is entirely isolated to `/app/` routes.

3. **Business logic wired in sequence** - consent service first (everything gates on this), then recruiter search, then recommendation gateway. Each time a service moves from stub to real, all pages consuming it automatically get live data.

The Key Rule

If a parallel builder finds themselves writing a condition like `if (user.tier === 'gold')` directly in their page component, that's a signal to stop and add a service function or extend `TierGate` instead. All conditional logic belongs in `/lib`, pages stay "dumb" consumers. This is what makes it safe to parallelise - the decision-making is centralised, the rendering is distributed.[osohq]

Can't parse the repos directly - GitHub is consistently blocking the tool (rate limiting or auth wall on the fetcher). I'll be transparent rather than guess at structure. What I can do is work through all the technical questions you raised directly, which don't depend on reading the repo.

---

3. On the Client Approval Gate

What Daniel described is almost certainly **branch protection rules** on `main` - first few pushes require his review via pull request, then possibly relaxed after trust is established. This is a sensible trust signal but creates a real risk for your workflow:[youtube]

1. **Stale DB probability increases** - if your team raises a PR with a Prisma schema change, and Daniel takes 24-48h to approve, everyone else has already pulled `main` without that schema change. They're now building against a DB shape that doesn't match what's about to land. With no CI/CD gate to catch it, this surfaces as a silent local break.
2. **Deployment isn't the same as merge** - "deployed after approval" likely means he has a webhook or manual step that redeploys the UCL VM on `main` push. That's fine for production but doesn't help local dev coordination at all.

Recommendation: Treat client approval gate as a `main` → production pipeline concern only. Your team should be working on a `dev` branch (or per-component branches merging into `dev`), and only raising PRs to `main` at defined integration checkpoints - not on every commit. This decouples your development velocity from Daniel's review cadence entirely.

On "No Extra Forks, Push to Sub-Main Branch"

What he likely means: a single repo with a `dev` or `team-1` branch sitting below `main`. Not technically a fork - just a branch. This is the cleaner model and avoids the fork-upstream-sync problem. Structurally:

text
```
main          ← Daniel controls, deploys to UCL VM on merge
```

```
└── dev       ← your team's integration branch
    ├── feat/auth-setup
    ├── feat/student-profile
    ├── feat/consent-skeleton
    └── feat/db-schema
```

You merge feature branches into `dev` freely, then raise one clean PR from `dev → main` at integration milestones. Daniel only sees/approves the milestone PR, not every team commit.[youtube]

On Kubernetes + PostgreSQL Stale Data Risk

This is a real but manageable risk, specifically two distinct issues:

1. Local dev DB drift (schema stale) - already covered in your migration runbook. Each developer runs `npx prisma migrate dev` after every pull that touches `schema.prisma`. The risk is low because you're on dummy data - a missing column causes a clear error, not silent corruption.

2. Kubernetes deployment stale environment variables - this is the subtler risk. When the UCL VM runs the app in a Kubernetes pod, env vars (database connection string, auth secrets, `NODE_ENV`) are injected at pod startup from a ConfigMap or env file. If two developers push changes that expect different env var shapes (e.g. one adds a `RECOMMENDATION_FEATURE_FLAG` var that the Kubernetes config doesn't have), the pod starts but the feature silently falls back or throws at runtime.[developers.googleblog]

Mitigation: maintain a `.env.example` file committed to the repo as the single source of truth for all expected env vars. Any new var added by any team member must be added to `.env.example` in the same commit - schema owner reviews this alongside Prisma changes. Local `.env` is gitignored as normal, but `.env.example` is always current.

bash
```bash
# .env.example - committed, always up to date
DATABASE_URL=postgresql://user:pass@localhost:5432/talent_dev
NEXTAUTH_SECRET=your-secret-here
NEXTAUTH_URL=http://localhost:3000
UCL_SSO_ENDPOINT=
RECOMMENDATION_FEATURE_FLAG=false
```

On Cloning the Whole Scaffold vs Fork

Don't fork. A fork creates a separate GitHub repo under your org or personal account, which means you'd need to manage upstream syncing manually every time Daniel pushes to the scaffold. Since Daniel confirmed direct collaboration on the scaffold repo, cloning and working in branches within that repo is strictly cleaner.[youtube]

The workflow is: clone once, create `dev` branch, everyone works in feature branches off `dev`, never directly on `main`. One sentence summary for the team: **we clone the scaffold, we don't fork it, and nobody pushes directly to main.**

What to Confirm with Daniel

Three quick asks that will resolve all the ambiguity:

1. Does `main` have branch protection rules requiring PR review? If yes, is he open to exempting a `dev` branch from that gate?
2. Does the UCL VM deployment trigger on every `main` push, or is it a manual deploy step?
3. Where does the Kubernetes ConfigMap / env var config live - in the repo or managed separately by him?

These are 5-minute clarifications that meaningfully de-risk your entire deployment model.